

CONFIDENTIAL · ENGINEERING AUDIT

Engineering Audit

theprofessionalnetwork

Full-Stack Architecture & Production Readiness Review

5.5

OVERALL

4.0

PROD READY

3.5

SECURITY

6.0

MAINTAINABILITY

1.0

TESTING

REPOSITORY

theprofessionalnetwork

BRANCH AUDITED

rayyan @ 63e0573

DATE

2026-05-13

STACK

Next.js 16 · React 19 · Tailwind
v4

APPLICATIONS

Marketing Site + SmartOffice
Dashboard

AUDITOR

Senior Full-Stack Architecture
Review

Table of Contents

01	Executive Summary	4
02	Engineering Scorecard	4
03	Repository & Architecture Overview	5
04	Key Strengths	6
05	Security Findings	7
06	Performance & Frontend Findings	8
07	Reliability & Error Handling Findings	9
08	Code Quality & Maintainability Findings	10
09	Testing & Reliability Assessment	10
10	DevOps & Deployment Assessment	11
11	Technical Debt Overview	11
12	Priority Action Table	12
13	Final Engineering Assessment	13

1 · Executive Summary

This audit covers the **theprofessionalnetwork** repository, which contains two distinct Next.js 16 applications co-located in a single git repository without formal monorepo tooling:

APPLICATION	PATH	PURPOSE	STAGE
Marketing Site	/ (root)	Public-facing landing & membership page	Early Prototype
SmartOffice	/smartoffice/	Internal event & member management dashboard	Active Development

The marketing site is a static, frontend-only application with no backend integration. Its primary user interaction — the membership application form — does not submit data anywhere. Several routes are unreachable due to a redirect component that forces all navigation to the homepage.

SmartOffice is a more mature application featuring email/OTP authentication, role-based access control, Redux state management, and a well-structured service layer. However, it has a critical SSRF vulnerability in its file download proxy, no server-side route protection, and no automated test coverage.

Both applications share zero automated tests, no CI/CD pipeline, and no error monitoring. The most urgent engineering investment is addressing the security vulnerabilities and adding server-side middleware authentication before expanding the user base.

2 · Engineering Scorecard

DIMENSION	SCORE		RATING	NOTES
Overall Engineering	5.5 / 10		FAIR	Solid foundations with significant gaps
Production Readiness	4.0 / 10		POOR	Multiple production blockers unresolved
Architecture	5.0 / 10		FAIR	No monorepo tooling; no API layer in marketing site
Security	3.5 / 10		POOR	Critical SSRF; localStorage JWT; no server-side auth
Performance	5.0 / 10		FAIR	Dual animation libraries; no image optimization
Maintainability	6.0 / 10		ADEQUATE	SmartOffice well-structured; marketing site is not

DIMENSION	SCORE		RATING	NOTES
Scalability	5.5 / 10		FAIR	Service layer scales; auth layer does not
Testing & Reliability	1.0 / 10		CRITICAL	Zero automated tests across both applications
DevOps & Observability	2.0 / 10		POOR	No CI/CD, monitoring, or error tracking
Developer Experience	5.5 / 10		FAIR	Good local patterns; no shared tooling or docs

3 · Repository & Architecture Overview

Repository Structure

The repository contains two independent Next.js 16 applications with no shared packages, no workspace tooling, and no unified CI configuration.

REPOSITORY LAYOUT

```
theprofessionalnetwork/ (git root)
├── app/          ← Marketing Site pages (Next.js App Router)
├── components/   ← 23 React components (all static/animation)
├── data/         ← Hardcoded professors.js, faqdetails.js
├── package.json  ← Marketing site dependencies only
└── smartoffice/ ← Separate Next.js app (independent npm install)
    ├── src/app/  ← Dashboard pages + 1 API route
    ├── src/store/ ← Redux + Zustand state
    ├── src/services/ ← Axios service layer
    ├── src/hooks/ ← Custom hooks
    └── package.json ← SmartOffice dependencies
```

Pseudo-Monorepo Risk: Two independent Next.js apps share one git repository without Turborepo or pnpm workspaces. Shared code must be copy-pasted. Dependency versions (React 19, Next.js 16, Tailwind v4) are duplicated and can silently diverge. A unified CI pipeline requires custom scripting.

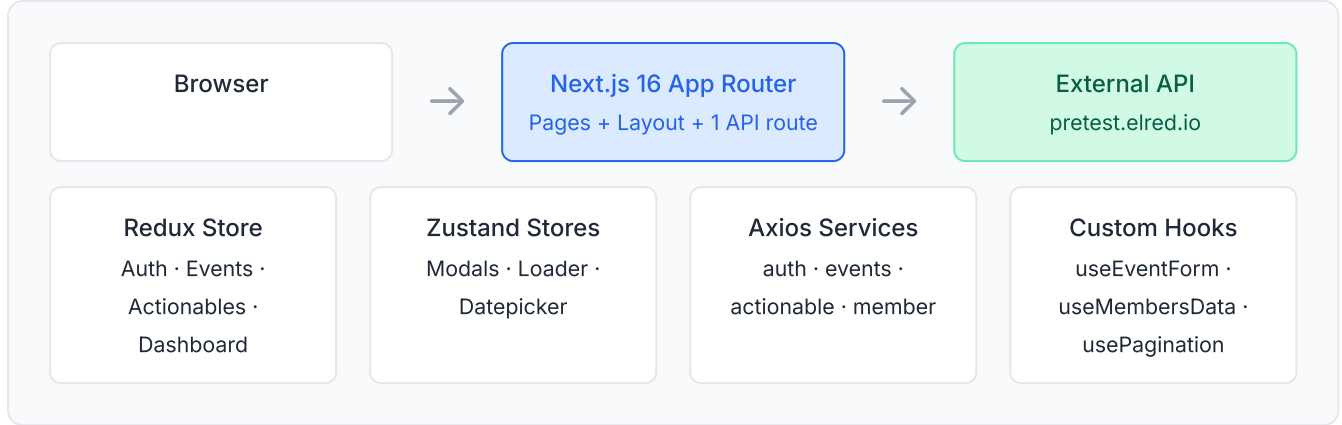
Marketing Site Architecture

The marketing site is a **static, frontend-only application** with no API routes, no backend, and no authentication. Notable architectural issues:

- **RefreshRedirect.jsx** forces all non-home routes to `/` — `/about` and `/faqs` are unreachable on direct navigation or hard refresh.
- **Custom window event bus** (`window.dispatchEvent(new Event('openApplyPopup'))`) is used for popup coordination with no guarantee listeners are registered before dispatch.
- **Application form is non-functional** — form state is collected but never submitted to any endpoint.
- Root `next.config.mjs` is entirely empty — no image domains, headers, or redirects configured.

SmartOffice Architecture

SmartOffice follows a clean layered architecture that separates concerns appropriately:



The mixed Redux (server data) + Zustand (UI state) strategy is appropriate. The primary architectural weakness is that authentication enforcement exists only at the client layer — there is no `middleware.js` running server-side to validate JWTs before rendering protected pages.

4 • Key Strengths

✓ Service Layer Consistency

All API calls route through a single Axios instance with a request interceptor for token attachment. Services follow a uniform pattern, making the API surface easy to navigate and extend.

✓ Optimistic UI Pattern

Task creation uses a `tempId` strategy — items appear immediately in the UI, are replaced with server IDs on success, and rolled back on failure. A well-implemented pattern that improves perceived performance.

✓ Clean Separation of Concerns

The `services/`, `hooks/`, `store/`, and `utils/` layering is applied consistently in SmartOffice. Business logic lives in dedicated utility modules rather than inside components.

✓ Custom Hooks

`useEventForm`, `useMembersData`, `useInfiniteScroll`, `usePagination`, and `useDebounce` are properly extracted. They encapsulate complex side-effect logic and keep components lean.

✓ Role-Based Access Control

Admin-only routes (`/checklist`, `/members`, `/events`) are defined in a central `routeAccess.js` and enforced in `ProtectedRoute`. Non-admins receive a 404 rather than an error page.

✓ Modern Stack

Both applications use Next.js 16 with React 19 and Tailwind CSS v4 — a forward-looking stack that avoids significant near-term upgrade debt. Formik + Yup provide structured form validation in SmartOffice.

✓ Pagination & Infinite Scroll

Members, events, and actionables all support server-side pagination with search and filter parameters. `useInfiniteScroll` and `usePagination` hooks are reused across features.

✓ Marketing Site Animation Quality

The marketing site uses both Framer Motion and GSAP ScrollTrigger to deliver polished, scroll-driven animations. The visual quality is high and reflects a strong design sensibility.

5 • Security Findings

HIGH SSRF Vulnerability — Unauthenticated File Download Proxy

FILE `smartoffice/src/app/api/download/route.js`

DETAIL The `/api/download?fileUrl=` endpoint fetches and streams any arbitrary URL with no authentication check, no authorization, and no URL validation. This is a Server-Side Request Forgery (SSRF) vector. An attacker who discovers this endpoint can use the server to fetch internal resources, probe internal networks, exfiltrate metadata, or bypass IP-based access controls.

FIX Validate `fileUrl` against an allowlist of trusted domains. Require a valid Bearer token. Add rate limiting.

```
// Minimum viable fix
const ALLOWED = ['res.cloudinary.com', 'assets-pretest.elred.io'];
const url = new URL(fileUrl);
if (!ALLOWED.includes(url.hostname)) {
  return new NextResponse('Forbidden', { status: 403 });
}
```

HIGH JWT Tokens Stored in localStorage

FILES `smartoffice/src/store/auth/authSlice.js`, `smartoffice/src/services/axios.js`,
`smartoffice/src/utils/auth.js`

DETAIL Authentication tokens are read from and written to `localStorage`. Any JavaScript executing on the page — including third-party analytics or ad scripts — can read and exfiltrate these tokens. This is the canonical XSS → session hijack attack vector.

FIX Migrate to `httpOnly` cookies issued by the backend. This makes tokens inaccessible to JavaScript entirely. At minimum, implement a strict Content Security Policy to reduce XSS surface area.

HIGH No Server-Side Route Protection

FILE `smartoffice/src/_components/ProtectedRoute.jsx`

DETAIL All authentication and authorization enforcement exists in client-side React components. A user who manipulates Redux state in `localStorage` can bypass all UI route guards. The server will render any dashboard page for any request — there is no token validation before page delivery.

FIX Add a `middleware.js` at the SmartOffice root. Next.js Middleware runs on the server before any page renders and is the correct place for auth gates on `/dashboard/*`.

HIGH dangerouslySetInnerHTML Without Sanitization

FILE `components/ProfessorPopup.jsx`

DETAIL Professor `description` fields from `data/professors.js` are rendered with `dangerouslySetInnerHTML`. If this data is ever sourced from a database or CMS in the future, this becomes a stored XSS vector.

FIX Sanitize all HTML content with `DOMPurify` before rendering, or migrate to a Markdown renderer.

MEDIUM No 401 / Session Expiry Handling

FILE `smartoffice/src/services/axios.js`

DETAIL The Axios instance is configured with `validateStatus: () => true`, meaning 401, 403, and 5xx responses all "succeed" from Axios's perspective. A token expiry will not trigger a logout. Users remain stuck in a broken state with failing API calls.

FIX Add a response interceptor: on 401, call `logout()` and redirect to login. Handle 5xx with a global error toast.

MEDIUM Hardcoded Application Identifier

FILE `smartoffice/src/app/Login.jsx`

DETAIL `hashId: "elRed"` is hardcoded in login request payloads. This should be an environment variable to support multi-environment deployments without code changes.

FIX Move to `NEXT_PUBLIC_HASH_ID` environment variable.

6 · Performance & Frontend Findings

HIGH Dual Animation Libraries — ~100–150KB Bundle Overhead

FILES `components/Firstprinciple.jsx` (GSAP), all other animated components (Framer Motion)

DETAIL Both Framer Motion and GSAP are bundled in the marketing site. GSAP is used only for scroll-triggered stagger animations in a single component. The combined overhead adds 100–150KB to the client bundle on a page that is otherwise a static marketing site.

FIX Migrate the `Firstprinciple.jsx` GSAP animations to Framer Motion's `whileInView` and `staggerChildren` variants. Removes the GSAP dependency entirely.

HIGH No Next.js `<Image>` Component in Marketing Site

DETAIL The marketing site uses raw `<img>` tags throughout, bypassing Next.js's built-in image optimization, lazy loading, format conversion (WebP/AVIF), and responsive sizing. Professor photos, team photos, and background images load at full resolution on all devices and viewports.

FIX Replace `<img>` with `next/image` for all static content. Expected 30–60% reduction in image payload on mobile viewports.

MEDIUM No Caching Strategy in SmartOffice

DETAIL SmartOffice re-fetches member lists, event lists, and action items on every component mount. Navigating away and back triggers a full refetch. There is no `staleTime`, no `lastFetched` timestamp, and no cache invalidation strategy.

FIX Add TanStack Query with appropriate `staleTime` and `gcTime` per resource type, or implement Redux selector memoization with a `lastFetched` timestamp guard in thunks.

MEDIUM Client-Side Auth Hydration Flash

DETAIL SmartOffice hydrates auth state from `localStorage` inside `useEffect`. There is an inevitable render cycle where `isAuthenticated` is `false` before hydration completes, potentially flashing the login page before redirecting to the dashboard.

FIX Server-side middleware (see Security §5) eliminates this entirely — auth is resolved before the page is rendered.

MEDIUM moment.js Dependency (Maintenance Mode)

FILE `smartoffice/src/utils/dateUtils.js`

DETAIL `moment@2.30.1` is in long-term maintenance mode and adds ~67KB to the bundle.

FIX Replace with `date-fns` (tree-shakeable, ~5KB per function used) or native `Intl.DateTimeFormat`.

LOW Inconsistent Page Metadata

DETAIL The marketing site's `/about` and `/faqs` pages have no custom `<head>` metadata. All pages share the root layout's default title and Open Graph image, which limits SEO effectiveness and social sharing appearance.

FIX Export a `generateMetadata` function from each page file.

7 · Reliability & Error Handling

HIGH Non-Functional Application Form

FILE `components/ApplyPopup.jsx`

DETAIL The membership application form collects name, email, phone, company, role, and years of experience — but the data is managed in local state and never submitted anywhere. There is no API endpoint, no email service call, no form backend integration. Users who apply receive no confirmation and no data is captured.

FIX Integrate with an email API (Resend, SendGrid) or a form backend (Formspree, Typeform) as a minimum viable solution. A single POST to a serverless function routing to email takes less than a day.

HIGH RefreshRedirect Breaks All Routes

FILE `components/RefreshRedirect.jsx`

DETAIL `RefreshRedirect` is included in the root layout and uses `usePathname` to forcibly redirect any non-home route to `/`. This makes `/about` and `/faqs` entirely unreachable via direct navigation, shared links, or browser refresh. This is a production-blocking bug that also incurs an SEO penalty.

FIX Remove `RefreshRedirect.jsx` entirely. Implement a proper `not-found.js` page for 404 handling.

MEDIUM Axios validateStatus Swallows All Errors

FILE `smartoffice/src/services/axios.js`

DETAIL The Axios instance uses `validateStatus: () => true`, meaning all HTTP status codes including 401, 403, 404, and 500 are treated as resolved promises. Many Redux thunks do not inspect `response.status`, so API failures are silently swallowed. A 401 will not trigger a logout.

FIX Add a response interceptor that handles 401 (logout + redirect) and 5xx (global error toast) systematically. Reserve `validateStatus` only for specific endpoints that need non-2xx handling.

MEDIUM Fragile Window Event Bus

FILES `components/Navbar.jsx`, `components/Footer.jsx`, `components/JoinSection.jsx`

DETAIL Popup coordination uses `window.dispatchEvent(new Event('openApplyPopup'))`. There is no guarantee event listeners are registered before dispatch, no error if a listener is missing, and no way to pass data through the event. The pattern is impossible to trace with standard debugging tools.

FIX Replace with React Context or a minimal Zustand store. Zustand is already a dependency in SmartOffice and would be a natural fit.

LOW No Error Boundaries

DETAIL Neither application has React Error Boundaries wrapping key sections. A runtime error in any client component will crash the entire page rather than degrading gracefully.

FIX Add `error.js` files at the route segment level in the Next.js App Router. These act as automatic error boundaries for their segment.

8 · Code Quality & Maintainability

MEDIUM No TypeScript

DETAIL Both applications use plain JavaScript with no type annotations. SmartOffice has complex Redux state shapes, nested event/actionable objects, and multi-parameter thunks — all of which benefit significantly from TypeScript. Type errors in these areas are currently discovered at runtime in production.

FIX Migrate SmartOffice to TypeScript incrementally. Start with `services/` files and Redux slice state interfaces — these yield the highest return on investment and unblock IDE autocompletion for the most complex parts of the codebase.

MEDIUM Content Hardcoded in Source Files

FILES `data/professors.js`, `data/faqdetails.js`

DETAIL Professor bios (including raw HTML), FAQ content, and team member data are all embedded in JavaScript files. Adding or updating a professor requires a code change and a full deployment. Some professor descriptions contain inline HTML strings.

FIX For the marketing site's current scale, a headless CMS (Contentful, Sanity) or a simple JSON file served from a CDN decouples content updates from deployments. At minimum, extract to a structured JSON format outside the component tree.

LOW README Not Customized

DETAIL Both `README.md` files contain the standard `create-next-app` boilerplate. There is no documentation for local setup, environment variables, deployment, or project structure. New developers must rely on tribal knowledge.

FIX Add a `.env.example` file (no values, just variable names) to SmartOffice immediately. This is a 5-minute fix with meaningful onboarding impact.

9 · Testing & Reliability Assessment

Zero automated tests exist across both applications. This is the single highest-risk engineering gap for production sustainability. Any significant refactor or feature addition carries unquantified regression risk.

Specific untested areas with high business impact:

- Authentication flow (email input → OTP → dashboard redirect)
- Optimistic update and rollback logic in `actionableSlice` thunks
- Route protection logic in `ProtectedRoute` and `GuestRoute`
- Form validation in `smartoffice/src/utils/validation.js`
- Event payload construction in `eventPayload.js`

- Role-based route access enforcement

Recommended Testing Roadmap

PRIORITY	TEST TYPE	TARGET	TOOL	ROI
1	Unit	validation.js, eventPayload.js, auth.js utilities	Vitest	High — no DOM required, fast to write
2	Integration	Redux thunks (actionable, events)	Vitest + MSW	High — protects optimistic update logic
3	Component	ProtectedRoute, GuestRoute auth redirect	React Testing Library	High — guards against auth regressions
4	E2E Smoke	Login → dashboard → create task flow	Playwright	Medium — catches integration failures

10 · DevOps & Deployment Assessment

No CI/CD pipeline exists. Deployments appear to be triggered manually via Vercel git integration. There are no GitHub Actions workflows, no `vercel.json`, and no automated quality gates before code reaches production.

GAP	IMPACT	EFFORT TO FIX
No CI build validation	Broken builds can reach production undetected	Low (GitHub Actions template)
No lint enforcement in pipeline	ESLint is configured but not enforced	Low (add to CI workflow)
No preview deployment per PR	Reviewers cannot test changes before merge	Low (Vercel auto-preview)
No error monitoring	Production errors invisible without user reports	Low (Sentry free tier)
No structured logging	Server-side errors leave no trace	Medium
Single backend environment	No staging/production separation	Medium (backend dependency)
No <code>.env.example</code>	New developer onboarding requires tribal knowledge	Trivial (5 minutes)

11 · Technical Debt Overview

CRITICAL localStorage JWT → httpOnly cookies migration	CRITICAL SSRF fix in download proxy (1 hour)	HIGH Zero test coverage — ongoing velocity risk	HIGH Non-functional application form
HIGH Broken route redirect (RefreshRedirect)	HIGH Server-side middleware for auth	MEDIUM Monorepo tooling (Turborepo)	MEDIUM TypeScript migration (SmartOffice services)
MEDIUM Next.js Image component adoption	MEDIUM Dual animation library consolidation	MEDIUM Caching strategy for SmartOffice data	LOW Replace moment.js with date-fns

12 · Priority Action Table

Immediate Priorities — Sprint 1

#	ACTION	FILE(S)	RISK IF DEFERRED
1	Secure <code>/api/download</code> — add domain allowlist + auth check	<code>smartoffice/src/app/api/download/route.js</code>	Active SSRF vulnerability accessible to any user
2	Fix <code>RefreshRedirect</code> — remove component, add <code>not-found.js</code>	<code>components/RefreshRedirect.jsx</code>	All <code>/about</code> and <code>/faqs</code> links broken
3	Handle 401 in Axios interceptor → auto-logout + redirect	<code>smartoffice/src/services/axios.js</code>	Silent auth failures; users stuck in broken state
4	Add Next.js Middleware for dashboard route auth	New: <code>smartoffice/src/middleware.js</code>	Client-side auth bypass; server renders protected pages
5	Wire up application form submission (email API or form backend)	<code>components/ApplyPopup.jsx</code>	Membership applications lost — core business flow broken
6	Add <code>.env.example</code> with all required variable names	New: <code>smartoffice/.env.example</code>	Onboarding friction; developers need tribal knowledge

Near-Term Improvements — Sprint 2-4		
#	ACTION	BENEFIT
7	Add GitHub Actions CI (build + lint on every PR)	Catch broken builds before merge
8	Migrate JWT storage to <code>httpOnly</code> cookies	Eliminate XSS-based token theft vector
9	Sanitize <code>dangerouslySetInnerHTML</code> with DOMPurify	Prevent stored XSS as content source evolves
10	Replace <code></code> tags with <code>next/image</code> in marketing site	30–60% image payload reduction on mobile
11	Remove GSAP, consolidate animations on Framer Motion	~80KB bundle reduction; single animation dependency
12	Add unit tests for utility functions (Vitest)	First test coverage; fastest ROI; no DOM setup needed
13	Configure Vercel preview deployments per branch	Enable PR testing without local setup
14	Replace window event bus with Zustand in marketing site	Traceable popup state; eliminates silent dispatch failures

Long-Term Enhancements — Quarter 2+		
#	ACTION	BENEFIT
15	Adopt Turborepo for monorepo management	Shared packages, unified CI, dependency consistency
16	TypeScript migration (start with SmartOffice services layer)	Eliminate runtime type errors; improve IDE support and DX
17	Add E2E test suite for critical flows (Playwright)	Regression protection for auth and event creation
18	Integrate error monitoring (Sentry)	Visibility into production errors without user reports
19	Replace <code>moment.js</code> with <code>date-fns</code>	~67KB bundle reduction; actively maintained library
20	CMS integration for marketing site content	Content updates without code deploys
21	Token refresh flow + structured session management	Better UX on long sessions; prepares for multi-tab logout
22	Add TanStack Query caching to SmartOffice data fetching	Reduces redundant network calls; improves perceived performance

13 · Final Engineering Assessment

Is the marketing site production-ready?

No — two immediate blockers: broken
✗ route redirect and non-functional application form.

These are not edge cases — they are the primary user interactions the site is built around. Both are fixable within a single sprint.

Is SmartOffice production-ready?

Conditionally — core dashboard works,
~ but SSRF vulnerability and missing server-side auth must be resolved first.

The application functions correctly for its intended use case. The security issues should be addressed before expanding the user base beyond a controlled internal audience.

Is the architecture scalable?

SmartOffice's service/store/hooks
~ layering scales well. The auth architecture does not.

The localStorage-based auth pattern does not support server-side rendering, token refresh, or multi-tab logout. This must be addressed before significant user growth. The marketing site has no scalability concerns at its current scope.

Are there major blockers?

! Two: SSRF vulnerability in download proxy, and zero test coverage.

The SSRF requires an immediate fix before exposing the application to untrusted users. Zero test coverage means any significant change carries unquantified regression risk — this compounds over time.

Areas to Evolve Over Time

AREA	CURRENT STATE	TARGET STATE	WHY IT MATTERS
Authentication	localStorage JWT, client-side guards	httpOnly cookies, Next.js Middleware	Highest-leverage security investment; also fixes UX flash
Testing	Zero coverage	Unit + integration + E2E smoke	Prevents regression as complexity grows; enables safe refactors
Monorepo Tooling	Two apps, one repo, no tooling	Turborepo with shared packages	Prevents dependency divergence; enables shared UI components
Observability	No monitoring, no logging	Sentry error tracking, structured logs	Production issues currently invisible without user reports
Type Safety	Plain JavaScript	TypeScript (services, store first)	SmartOffice's Redux shapes are complex; type errors cost more at

AREA	CURRENT STATE	TARGET STATE	WHY IT MATTERS
			scale
Content Management	Hardcoded in JS files	Headless CMS or structured JSON	Content updates currently require code deployments

Summary: The engineering team has made sound architectural choices in SmartOffice's service and state management layers. The most impactful near-term investments are closing the security gaps (SSRF fix, server-side auth), fixing the two marketing site blockers, and establishing a testing baseline. These changes, achievable in two sprints, would substantially improve the production confidence and engineering sustainability of both applications.